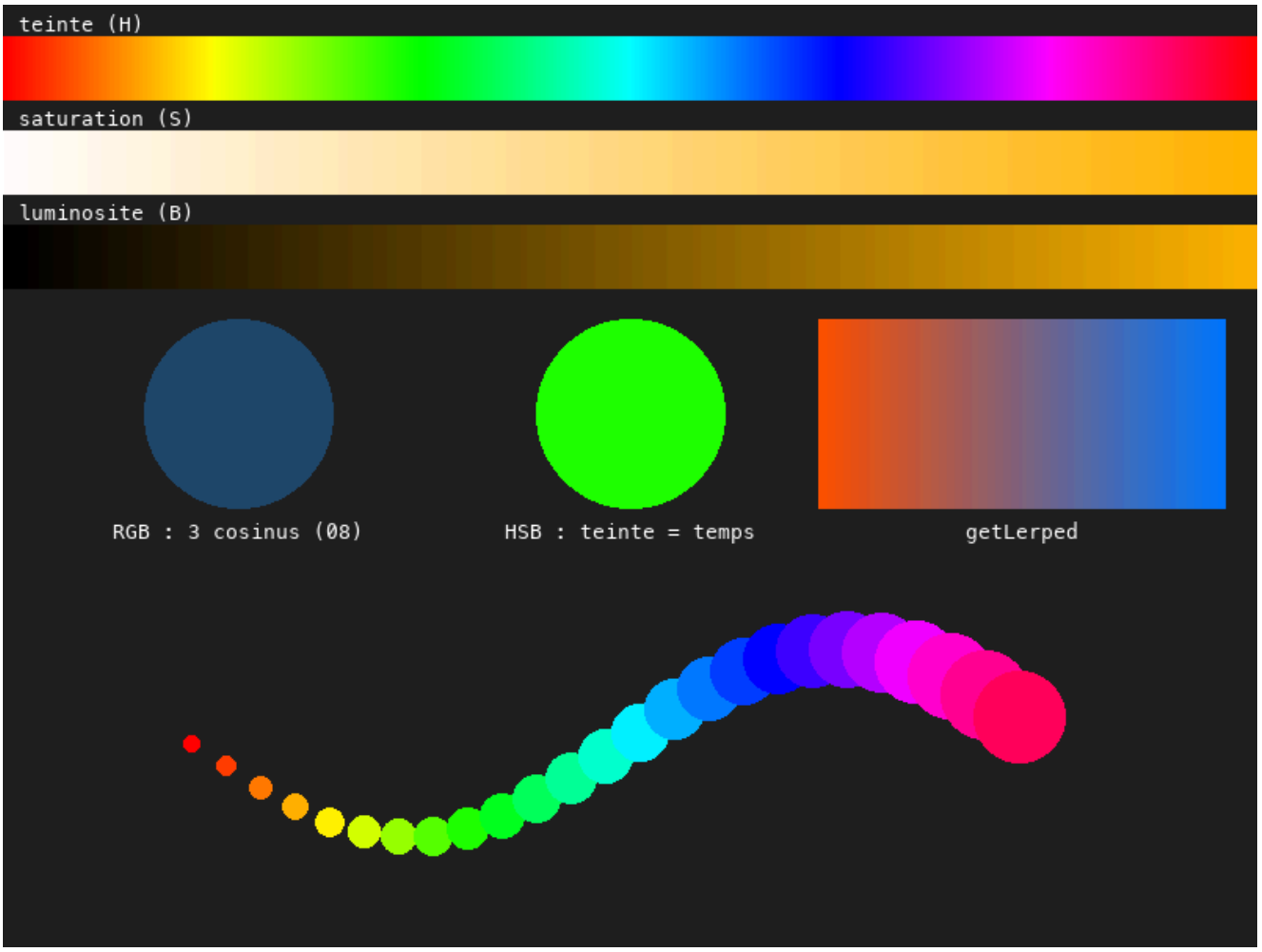
Cours 09 — Couleur : RGB, HSB, dégradés

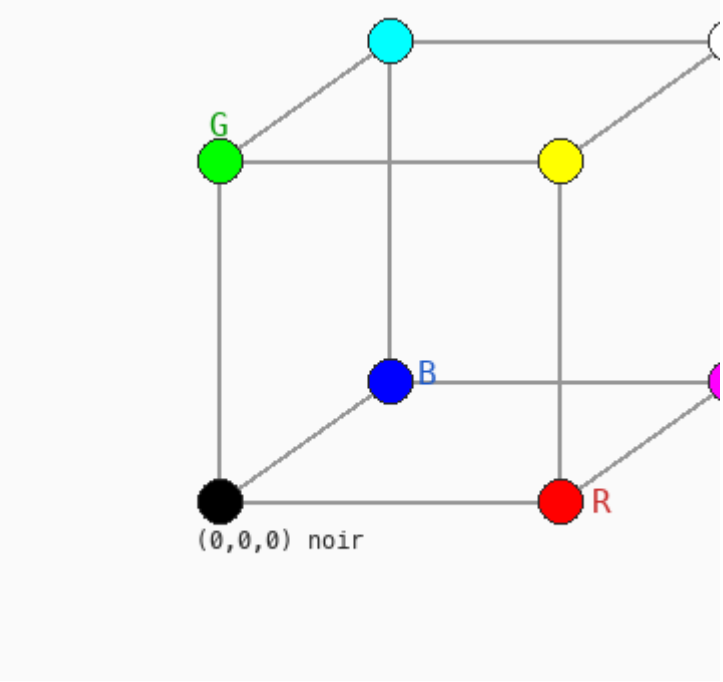
> **Avant** : cours 07 (traînée, couleur animée par cosinus). **Comment travailler** : toujours dans `ofApp.h` et `ofApp.cpp`, par versions successives. `ofApp09.h` / `ofApp09.cpp` : la référence téléchargeable de l'état final.

Depuis le cours 00, une couleur est trois nombres : rouge, vert, bleu. C'est ainsi que l'**écran** travaille — pas ainsi que **nous** pensons la couleur. Ce cours introduit une deuxième manière de la décrire, la teinte, et un type pour la manipuler comme une valeur : `ofColor`.



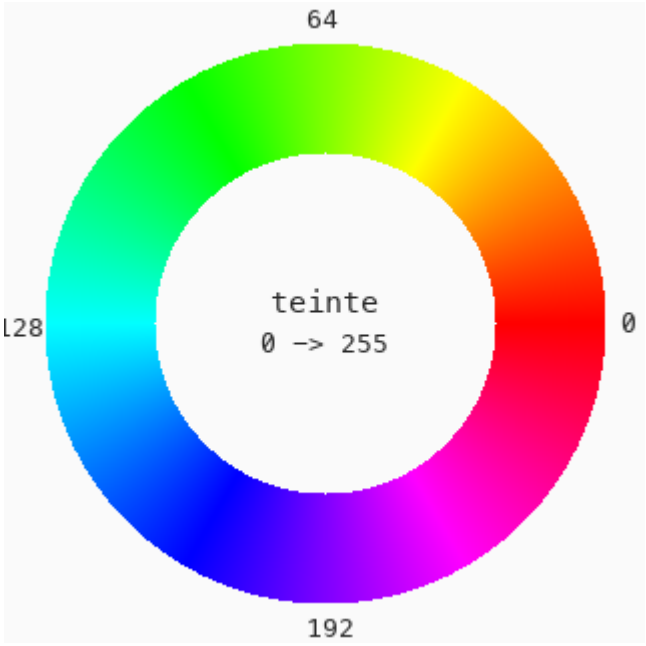
1. Deux façons de décrire une couleur

RGB : un cube. Une couleur RGB est un point dans un cube — trois axes, rouge, vert, bleu, chacun de 0 à 255. Le noir à l'origine, le blanc au sommet opposé. Pratique pour l'écran, pénible pour l'humain : « la même couleur, un peu plus claire » ou « la couleur suivante de l'arc-en-ciel » ne se calculent pas simplement dans ce cube. C'est pour ça que le cours 07 bricolait trois cosinus.



HSB : une roue. HSB décrit une couleur comme on la **nomme** :

Axe	Nom	0	255
H	**teinte** (hue)	rouge, puis en tournant : jaune, vert, cyan, bleu, magenta	et retour au rouge
S	**saturation**	gris	couleur pure
B	**luminosité** (brightness)	noir	couleur pleine



La teinte est une **position sur une roue** : après 255, on revient à 0. « La couleur suivante » devient « teinte + 1 » ; « plus clair » devient « luminosité + 1 ». Attention : en openFrameworks les trois axes vont de **0 à 255** (pas de degrés).

2. Étape 1 — voir les trois axes

Fenêtre 800 × 600, et dans `draw()` :

```
float largeur = ofGetWidth();
float teinteChoisie = mouseX / largeur * 255;   // la souris choisit la teinte

for (int x = 0; x < largeur; x++) {
        float p = x / largeur * 255;                // 0 à gauche, 255 à droite

        ofSetColor(ofColor::fromHsb(p, 255, 255));              // bande 1 : la teinte
        ofDrawLine(x, 20, x, 60);

        ofSetColor(ofColor::fromHsb(teinteChoisie, p, 255));    // bande 2 : la saturation
        ofDrawLine(x, 70, x, 110);

        ofSetColor(ofColor::fromHsb(teinteChoisie, 255, p));    // bande 3 : la luminosité
        ofDrawLine(x, 120, x, 160);
}
```

Trois bandes : la roue entière, puis gris → couleur pure, puis noir → couleur pleine. Bouge la souris : les bandes 2 et 3 changent de teinte. Une ligne verticale par pixel, et à chaque pixel **un seul axe varie** — on *voit* les axes.

Les nouveautés :

- `ofColor` est un type qui contient une couleur entière. C'est une `struct` (cours 07) : quatre variables `r`, `g`, `b`, `a` regroupées, accessibles avec un point (`c.r`). On peut la ranger dans une variable, la passer à `ofSetColor`, la transformer.
- `ofColor::fromHsb(h, s, b)` construit une couleur depuis ses axes HSB. Le `::` se lit « de la famille `ofColor` » — un outil rangé avec le type. Dans l'autre sens, `c.getHue()`, `c.getSaturation()`, `c.getBrightness()` lisent les axes d'une couleur existante.
- `ofDrawLine(x1, y1, x2, y2)` : un trait entre deux points.

> **Essaie** : inverse la première bande (`255 - p`). Puis ajoute une quatrième bande de ton invention — par exemple saturation **et** luminosité qui varient ensemble.

3. Étape 2 — animer une teinte

Sous les bandes, deux cercles côte à côte — l'ancien monde et le nouveau :

```
// la méthode du cours 07 : trois cosinus déphasés
float r = (cos(t * 1.2f)  / 2 + 0.5f) * 255;
float g = (cos(t * 1.0f)  / 2 + 0.5f) * 255;
float b = (cos(t * 0.86f) / 2 + 0.5f) * 255;
ofSetColor(r, g, b);
ofDrawCircle(150, 260, 60);

// en HSB : la teinte avance, saturation et luminosité pleines
float teinte = fmod(t * 40, 255);
ofSetColor(ofColor::fromHsb(teinte, 255, 255));
ofDrawCircle(400, 260, 60);
```

(Avec `t = ofGetElapsedTimef();` dans `update()`, comme au cours 07.) Laisse tourner et compare : à gauche ça bouge mais on ne contrôle rien — teintes sales, passages par le gris ; à droite, **toutes les couleurs pures, dans l'ordre**, sans jamais griser.

`fmod(a, b)` est le reste de la division pour les `float` : `t * 40` grandit sans fin, `fmod(t * 40, 255)` repart à 0 après 255 — la teinte fait le tour de la roue, encore et encore.

> **Essaie** : `t * 200` au lieu de `t * 40`. Puis calcule : à `t * 40`, combien de secondes dure un tour de roue complet ?

4. Étape 3 — la traînée arc-en-ciel

Reprends la file de 25 positions du cours 07 (`prevX`, `prevY`, remplies dans `update()`), et dessine :

```
for (int i = 0; i < prevX.size(); i++) {
          ofSetColor(ofColor::fromHsb(i * 10, 255, 255));
          ofDrawCircle(prevX[i], prevY[i], 5 + i);
}
```

La traînée évoluée du cours 07, en **une** ligne de couleur : la teinte vaut `i * 10`, la traînée est un arc-en-ciel. C'est le motif « nombre qui dépend de l'indice », avec le bon outil.

> **Essaie** : fais tourner l'arc-en-ciel avec le temps — `i * 10 + t * 40`. Il faudra `fmod` pour rester sur la roue.

5. Étape 4 — le dégradé : l'interpolation

```
ofColor depart(255, 80, 0);
ofColor arrivee(0, 120, 255);
for (int x = 520; x < 780; x++) {
        float p = (x - 520) / 260.0f;          // 0 → 1
        ofSetColor(depart.getLerped(arrivee, p));
        ofDrawLine(x, 200, x, 320);
}
```

`a.getLerped(b, p)` (*linear interpolation*) calcule la couleur à la proportion `p` entre les deux : `p = 0` donne `depart`, `p = 1` donne `arrivee`, `0.5` le milieu. Dans le cube RGB, c'est une **ligne droite** entre deux points.

Retiens le mot : **interpoler**. Il reviendra pour des positions, des tailles — tout ce qu'on veut faire passer en douceur d'une valeur à une autre.

Essaie : compare avec un dégradé fait en interpolant la **teinte** : `fromHsb(hDepart + (hArrivee - hDepart) * p, 255, 255)`. Lequel des deux passe par le gris, et pourquoi ?

Exercices

1. **Lire avant de lancer** — décris la bande produite par `ofColor::fromHsb(0, 255, p)` quand `p` va de 0 à 255. Puis celle de `fromHsb(p, 0, 255)`. Vérifie ensuite.
2. **La traînée apprivoisée** — reprends ta traînée évoluée du cours 07 et remplace les trois cosinus par une teinte qui avance (`fmod(t * 40, 255)`). Même vie, zéro gris.
3. **Les couleurs qui vont ensemble** — les cercles du cours 03, colorés de deux façons : teinte aléatoire (saturation et luminosité fixes) contre trois `ofRandom` sur r, g, b. Lequel donne une palette harmonieuse, et pourquoi ?
4. **Le coucher de soleil** *(plus costaud)* — un fond en dégradé **vertical** (une boucle sur `y`, des lignes horizontales, `getLerped` de l'orange au bleu nuit), et un soleil dont la teinte dépend de sa hauteur : plus la souris descend, plus il rougit.

Ce qu'il faut retenir

- RGB = le cube de l'écran ; HSB = la roue humaine : teinte, saturation, luminosité — 0 à 255 partout en openFrameworks.
- `ofColor` : la couleur comme **valeur** (une struct : `c.r`, `c.getHue()`…) ; `ofColor::fromHsb(h, s, b)` la construit depuis la roue.
- `fmod(a, b)` : le reste à virgule — l'outil des grandeurs qui tournent en rond, comme la teinte.

- Teinte selon le temps : une couleur qui vit sans griser ; teinte selon l'indice : un arc-en-ciel le long d'une file.
- `a.getLerped(b, p)` : **interpoler** — la valeur à la proportion `p` entre deux bornes.